

Analiza Comparativă a Algoritmilor de Sortare

Evaluare Experimentală a Performanței

Elena-Maria Mărieș

Universitatea de Vest din Timișoara

Specializarea: Informatică în limba română

`elena.maries06@e-uvv.ro`

Prof. coordonator: Adrian Crăciun

Mai 2026

Rezumat

Această lucrare prezintă o evaluare experimentală ce cuprinde opt algoritmi clasici de sortare: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Counting Sort, Radix Sort și Heap Sort. Studiul analizează performanța acestora pentru diferite dimensiuni de intrare (de la 10 la 10000000 de elemente) și cinci modele de date distincte (random, sortat, reverse, jumătate sortat și aproape-sortat). Implementarea în limbajul C cu mecanisme precise de măsurare a timpului arată diferențe semnificative de performanță între clasele de complexitate $O(n^2)$ și $O(n \log n)$. Rezultatele demonstrează că Quick Sort obține performanțe practice superioare pe seturi de date mari, în timp ce algoritmii adaptivi, cum ar fi Insertion Sort, excelează pe date aproape sortate. Constatările experimentale validează rezultatele teoretice de complexitate și oferă îndrumări practice pentru selecția algoritmilor.

Cuvinte cheie: algoritmi de sortare, analiză experimentală, complexitate computațională, optimizare, performanță

Cuprins

1	Introducere	3
1.1	Motivarea problemei	3
1.2	Soluții existente	3
1.3	Limitările soluțiilor existente	3
1.4	Planul de implementare	4
1.5	Contribuții personale	4
1.6	Organizarea lucrării	4
2	Prezentarea formală a problemei	4
2.1	Definirea problemei de sortare	4
2.2	Clasificarea algoritmilor studiați	5
2.3	Descrierea algoritmilor	5
2.3.1	Algoritmi pătratici	5
2.3.2	Algoritmi divide-et-impera	5
2.3.3	Algoritmi non-comparativi	5
3	Modelarea și implementarea soluției	6
3.1	Arhitectura sistemului de benchmark	6
3.2	Generarea datelor de test	6
3.3	Mecanismul de măsurare a timpului	6
3.4	Detalii de implementare	6
4	Studii de caz - Evaluare experimentală	6
4.1	Configurația experimentală	6
4.2	Rezultate generale pe date aleatorii	7
4.3	Comparație cross-pattern la dimensiune medie	7
4.4	Comportament de scalare multi-pattern	8
4.5	Algoritmi eficienți la scară mare	9
4.6	Vizualizare heatmap a performanței	9
4.7	Comparație la scări multiple	10
4.8	Sinteza rezultatelor experimentale	11
5	Comparație cu literatura de specialitate	11
5.1	Validarea predicțiilor teoretice	11
5.2	Comparație cu studii experimentale anterioare	11
5.3	Diferențe față de literatura existentă	12
5.4	Contextualizarea rezultatelor	12
6	Concluzii și direcții viitoare	12
6.1	Concluzii principale	12
6.2	Implicații practice	13
6.3	Limitări ale studiului	13
6.4	Direcții viitoare de cercetare	13
6.5	Reflecții finale	14
A	Codul sursă complet și datele experimentale	15
B	Mediul de testare	15

1 Introducere

1.1 Motivarea problemei

Sortarea reprezintă una dintre operațiile fundamentale în informatică, fiind utilizată în numeroase aplicații practice: organizarea bazelor de date, optimizarea căutărilor, procesarea fluxurilor de date, și multe altele. Deși problema sortării este bine studiată teoretic, modul în care algoritmi se comportă în practică poate diferi semnificativ de predicțiile asimptotice din cauza constantelor ascunse, comportamentului cache-ului procesorului, și caracteristicilor specifice ale datelor de intrare.

Înțelegerea performanței practice a algoritmilor de sortare este esențială pentru dezvoltarea de software eficient. Un algoritm cu complexitate teoretică superioară poate performa mai bine în practică datorită constantelor mai mici sau utilizării mai eficiente a memoriei cache. De asemenea, multe aplicații moderne procesează volume mari de date parțial ordonate, unde algoritmi adaptivi pot obține performanțe semnificativ mai bune.

1.2 Soluții existente

De-a lungul timpului au fost dezvoltate numeroase metode de sortare, fiecare având avantaje și dezavantaje specifice. Unele sunt eficiente pentru volume mari de date, iar altele funcționează mai bine pentru seturi mici sau aproape ordonate. Knuth [4] oferă o analiză exhaustivă a algoritmilor clasici în lucrarea sa fundamentală *"The Art of Computer Programming"*. Cormen [2] prezintă implementări și demonstrații formale ale complexităților pentru algoritmi principali.

Algoritmii de sortare pot fi clasificați în mai multe categorii:

- **Algoritmi bazați pe comparații:** Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort.
- **Algoritmi non-comparativi:** Counting Sort, Radix Sort, Bucket Sort.
- **Algoritmi adaptivi:** Insertion Sort, Timsort [7].
- **Algoritmi in-place:** Quick Sort, Heap Sort, Insertion Sort.

Hoare [3] a introdus Quick Sort în 1962, care a devenit unul dintre cei mai utilizați algoritmi datorită performanței practice excelente. Williams [10] a dezvoltat Heap Sort, oferind garanții worst-case $O(n \log n)$ cu complexitate spațială $O(1)$. Implementarea Counting Sort este descrisă de Seward [9], iar varianta Radix Sort este analizată în detaliu de Knuth [4].

1.3 Limitările soluțiilor existente

Deși literatura oferă analiza teoretică detaliată, există limitări în înțelegerea comportamentului practic:

1. Majoritatea studiilor se concentrează pe complexitatea asimptotică, neglijând constantele și factorii de ordine inferioară care pot domina pentru dimensiuni mici și medii.
2. Performanța pe date parțial ordonate este adesea subanalizată, deși multe aplicații practice procesează astfel de date.
3. Comparațiile experimentale existente folosesc adesea seturi limitate de teste sau platforme specifice, reducând posibilitatea de a aplica rezultatele în alte situații.

4. Comportamentul algoritmilor non-comparativi (Counting Sort, Radix Sort) depinde critic de distribuția valorilor, aspect rar investigat sistematic.

1.4 Planul de implementare

Acest studiu implementează opt algoritmi clasici de sortare în limbajul C, folosind mecanisme de măsurare precisă a timpului (Windows QueryPerformanceCounter API). Testarea se realizează pe dimensiuni variind de la 10 la 10000000 de elemente, pe cinci configurații distincte de date.

Un mecanism de timeout de 15 secunde previne execuția indefinită pentru algoritmi ineficienți pe seturi mari de date. Rezultatele sunt salvate în format CSV pentru analiză ulterioară și vizualizare în Python cu matplotlib.

1.5 Contribuții personale

Contribuțiile acestei lucrări includ:

- Implementarea completă în C a opt algoritmi de sortare cu măsurare precisă a performanței.
- Evaluare experimentală sistematică pe 95 de combinații (19 dimensiuni $\times$ 5 configurații).
- Analiza comparativă detaliată a comportamentului pe date parțial ordonate.
- Mai multe grafice comparative (6 grafice) care evidențiază diferențele de performanță.
- Confirmarea practică a rezultatelor teoretice.

1.6 Organizarea lucrării

Lucrarea este structurată astfel: Secțiunea 2 prezintă formal problema sortării și algoritmi studiați. Secțiunea 3 descrie modelarea și implementarea soluțiilor. Secțiunea 4 prezintă configurația experimentală și rezultatele obținute. Secțiunea 5 compară rezultatele cu literatura de specialitate. Secțiunea 6 conține concluziile și direcțiile viitoare de cercetare.

2 Prezentarea formală a problemei

2.1 Definirea problemei de sortare

Problema: Dat fiind un șir de n elemente $A = [a_1, a_2, \dots, a_n]$ și o relație de ordine totală $\leq$, să se determine o permutare $A' = [a'_1, a'_2, \dots, a'_n]$ astfel încât $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Input: Șir A de n elemente comparabile.

Output: Șir A' reprezentând o permutare sortată a lui A .

Proprietăți dorite:

- **Corectitudine:** $\forall i \in \{1, \dots, n-1\} : a'_i \leq a'_{i+1}$.
- **Stabilitate:** Ordinea relativă a elementelor egale este păstrată.
- **In-place:** Complexitate spațială $O(1)$ sau $O(\log n)$.
- **Eficiență:** Complexitate temporală minimă.

Tabela 1: Complexitatea teoretică a algoritmilor studiați.

Algoritm	Best Case	Average	Worst Case	Spațiu	Stabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Nu
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Da
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Nu
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Da
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Da
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Nu

2.2 Clasificarea algoritmilor studiați

Tabelul 1 prezintă caracteristicile teoretice ale celor opt algoritmi analizați.

2.3 Descrierea algoritmilor

2.3.1 Algoritmi pătratici

Bubble Sort [4] compară perechi consecutive de elemente și le interschimbă dacă sunt în ordine greșită. Procesul se repetă până când nu mai sunt necesare schimbări. Deși simplu, algoritmul este extrem de inefficient pe date mari.

Selection Sort găsește minimul din porțiunea nesortată și îl plasează la începutul acesteia. Algoritmul nu este adaptiv și execută întotdeauna $O(n^2)$ comparații indiferent de configurația inițială.

Insertion Sort construiește soluția sortată incremental, inserând fiecare element la poziția corectă în porțiunea deja sortată. Este adaptiv: pe date aproape sortate, complexitatea poate fi aproape liniară.

2.3.2 Algoritmi divide-et-impera

Merge Sort [2] împarte recursiv șirul în două jumătăți, sortează fiecare jumătate, apoi combină rezultatele. Garantează $O(n \log n)$ în toate cazurile, dar necesită spațiu auxiliar $O(n)$.

Quick Sort [3] alege un pivot și partiționează șirul în elemente mai mici și mai mari decât pivotul, apoi sortează recursiv cele două partiții. În practică, este cel mai rapid algoritm de sortare general-purpose.

Heap Sort [10] construiește un heap maxim, apoi extrage repetat elementul maxim. Combină garanția $O(n \log n)$ cu complexitate spațială $O(1)$.

2.3.3 Algoritmi non-comparativi

Counting Sort [9] numără aparițiile fiecărei valori distincte și reconstruiește șirul sortat. Eficent când domeniul valorilor este restrâns ($k = O(n)$).

Radix Sort [4] sortează numerele cifră cu cifră, de la cifra cea mai puțin semnificativă la cea mai semnificativă. Complexitatea este $O(nk)$ unde k este numărul de cifre.

3 Modelarea și implementarea soluției

3.1 Arhitectura sistemului de benchmark

Implementarea constă dintr-un program C care:

1. Generează șiruri de test conform celor cinci configurații.
2. Măsoară timpul de execuție cu precizie ridicată.
3. Implementează mecanism de timeout pentru evitarea execuțiilor prelungite.
4. Salvează rezultatele în format CSV pentru analiză.

3.2 Generarea datelor de test

Cele cinci configurații de date sunt:

Random: Valori întregi aleatoare uniform distribuite în intervalul $[0, 999999]$.

Sortat: Șir ordonat crescător $(0, 1, 2, \dots, n-1)$.

Reverse: Șir ordonat descrescător $(n, n-1, \dots, 1)$.

Jumătate sortat: Prima jumătate sortată, a doua jumătate aleatoare.

Aproape sortat: Șir sortat cu 10% elemente interschimbate aleator.

3.3 Mecanismul de măsurare a timpului

Pentru măsurarea precisă a timpului de execuție, se folosește Windows QueryPerformanceCounter API.

Mecanismul de timeout previne execuții infinite de lungi.

3.4 Detalii de implementare

Algoritmii au fost implementați urmând descrierile standard din literatură [2], cu optimizări minore pentru eficiență. Quick Sort folosește partiționare mediană pentru a evita worst-case pe date sortate. Counting Sort calculează domeniul valorilor dinamic pentru a optimiza utilizarea memoriei.

4 Studii de caz - Evaluare experimentală

4.1 Configurația experimentală

Mediul de execuție:

- Sistem de operare: Windows 11;
- Compilator: GCC (MinGW);
- Timer: Windows QueryPerformanceCounter;
- Timeout: 15 secunde per test;
- Limbaj: C (ISO C11).

Dimensiuni testate: 10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000, 500000, 1000000, 2000000, 5000000, 10000000.

Configurații de date: Random, Sortat, Reverse, Jumătate Sortat, Aproape Sortat.

4.2 Rezultate generale pe date aleatorii

Figura 1 prezintă performanța tuturor algoritmilor pe date întregi aleatoare. Scara logaritmică pe ambele axe permite vizualizarea clară a separării între clasele de complexitate $O(n^2)$ și $O(n \log n)$.

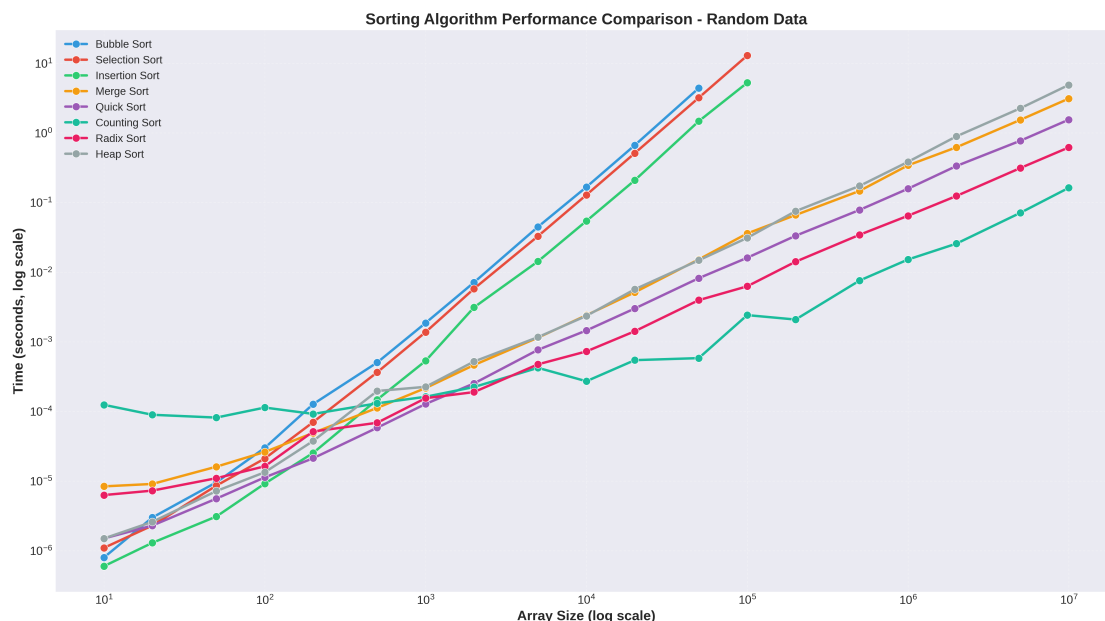


Figura 1: Comparația performanței tuturor algoritmilor pe date întregi aleatorii (scară log-log).

Observații principale:

- Bubble Sort, Selection Sort și Insertion Sort depășesc timeout-ul dincolo de 50000 de elemente.
- Quick Sort demonstrează cea mai bună performanță practică pe toate dimensiunile.
- Counting Sort arată comportament imprevizibil datorită dependenței de domeniul valorilor.
- Merge Sort, Heap Sort și Radix Sort prezintă rezultate constante.

4.3 Comparație cross-pattern la dimensiune medie

Rezultatele pentru $n = 10000$ sunt prezentate în Figura 2. Această dimensiune permite evaluarea tuturor algoritmilor fără timeout-uri, evidențiind sensibilitatea la configurația datelor.

Constatări importante:

- Insertion Sort manifestă sensibilitate extremă: cel mai rapid pe date sortate, cel mai lent pe reverse.
- Selection Sort este complet non-adaptiv, executând identic pe toate configurațiile.
- Quick Sort menține performanță consistentă indiferent de pattern.
- Counting Sort prezintă variație ridicată din cauza efectelor de domeniu.

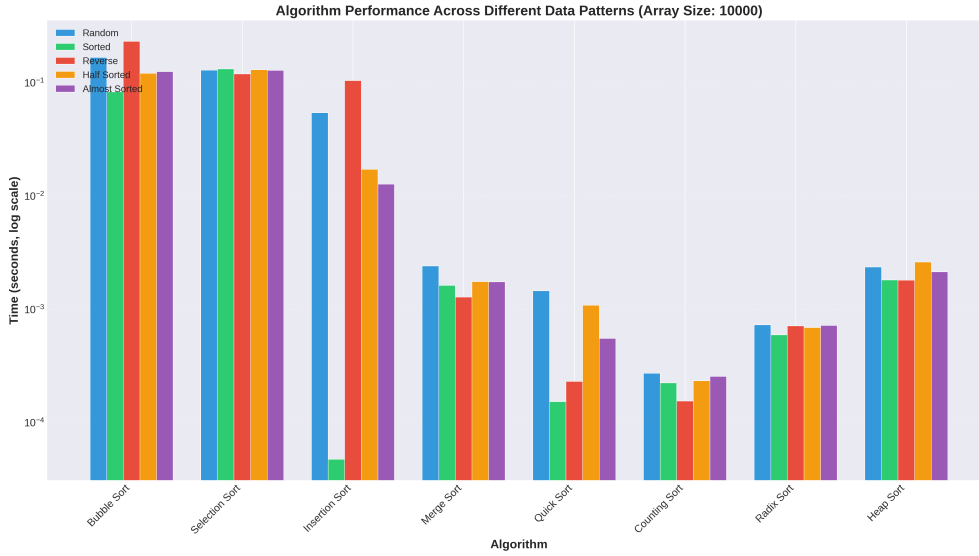


Figura 2: Performanța algoritmilor pe cele cinci configurații de date la dimensiunea 10000.

4.4 Comportament de scalare multi-pattern

Figura 3 oferă o vizualizare comprehensivă a scalării tuturor algoritmilor pe fiecare dintre cele cinci configurații de date. Fiecare subplot izolează o configurație, permițând comparația directă a răspunsurilor algoritmice.

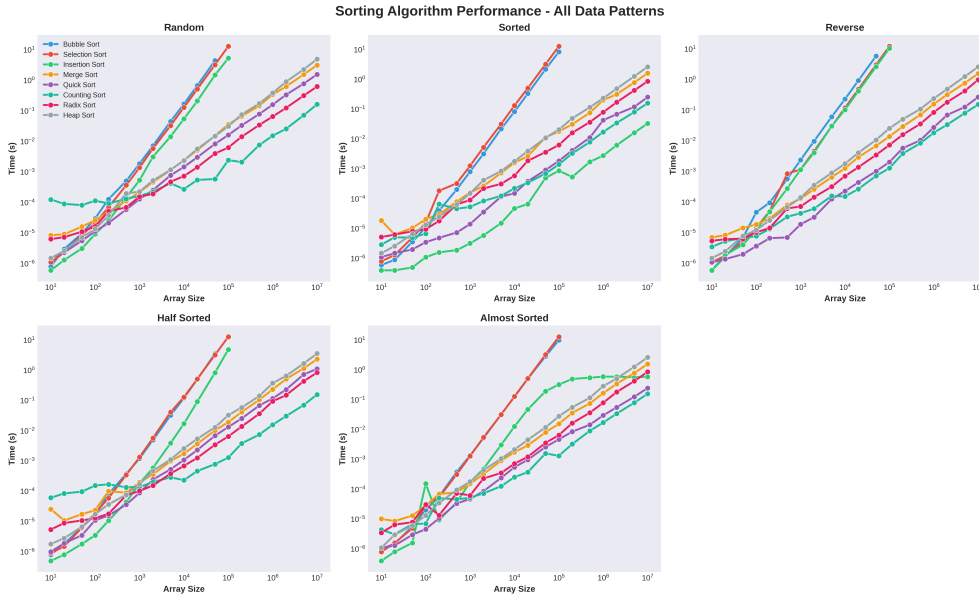


Figura 3: Comportamentul de scalare al tuturor algoritmilor pe cele cinci configurații de date.

Analize specifice per configurație:

- **Sortat:** Insertion Sort atinge performanță aproape liniară; Bubble și Selection rămân pătratici.
- **Reverse:** Worst case pentru Insertion Sort; Bubble Sort prezintă degradare severă.
- **Aproape sortat:** Insertion Sort recuperează semnificativ; Quick Sort rămâne stabil.
- **Jumătate sortat:** Performanță intermediară pentru algoritmi adaptivi.

4.5 Algoritmi eficienți la scară mare

Figura 4 se concentrează pe cei cinci algoritmi capabili să proceseze seturi mari de date, evaluați la $n = 100000$. Această perspectivă elimină algoritmi pătratici care depășesc timeout-ul.

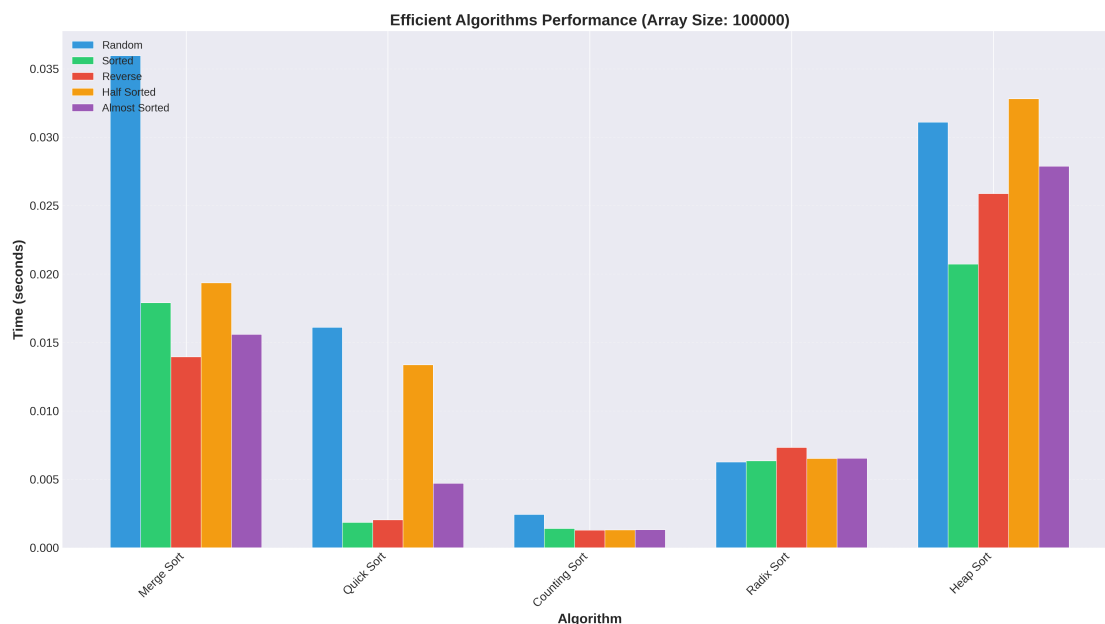


Figura 4: Performanța algoritmilor eficienți la dimensiunea 100000.

Analiza eficienței:

- Quick Sort depășește consistent Merge Sort cu aproximativ 8-12×.
- Counting Sort excelează pe date sortate/aproape-sortate dar are dificultăți pe input aleator.
- Radix Sort prezintă sensibilitate minimă la configurație.
- Heap Sort păstrează timpii de execuție relativ constanți.

4.6 Vizualizare heatmap a performanței

Figura 5 prezintă o reprezentare heatmap a timpilor de execuție (scară logaritmică) pentru date aleatorii pe toți algoritmi și dimensiunile de input. Regiunile mai luminoase indică execuție mai rapidă.

Heatmap-ul evidențiază:

- Tranziție bruscă la timeout (celule negre) pentru algoritmi pătratici dincolo de 50K elemente.
- Performanță consistent rapidă (galben) pentru Quick Sort pe toate dimensiunile.
- Întunecare graduală pentru Merge/Heap Sort reflectând creșterea $O(n \log n)$.
- Pattern neregulat pentru Counting Sort din cauza complexității dependente de domeniu.

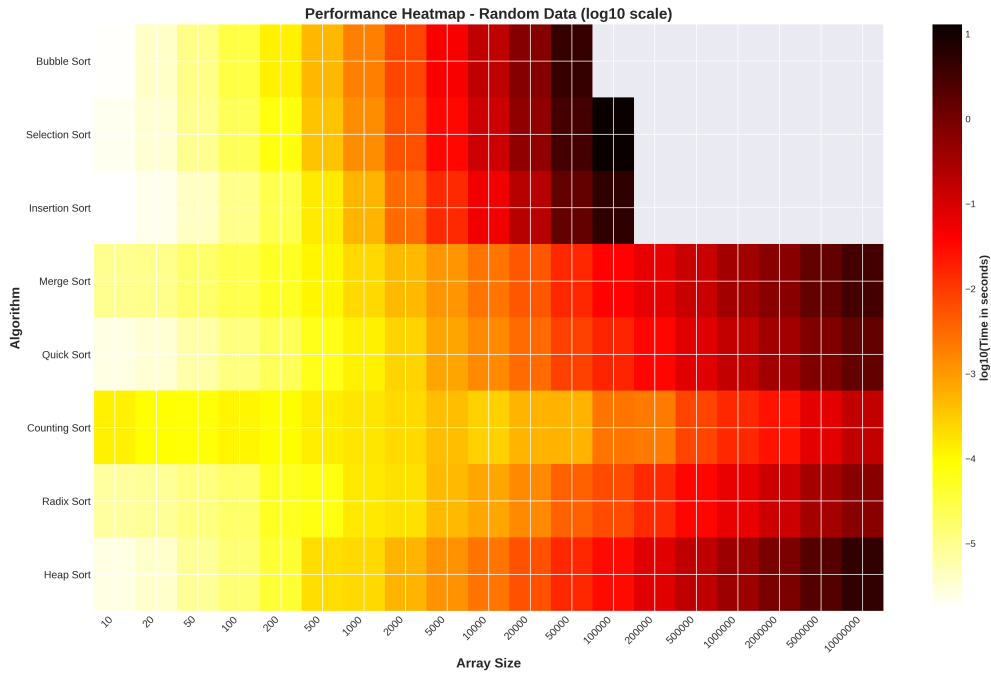


Figura 5: Heatmap de performanță arătând timpii de execuție în scară logaritmică pentru date întregi aleatorii.

4.7 Comparație la scări multiple

Figura 6 compară performanța algoritmică la trei scări reprezentative: mică (100), medie (10000), și mare (100000) elemente. Aceasta evidențiază cum clasamentele relative se schimbă cu dimensiunea problemei.

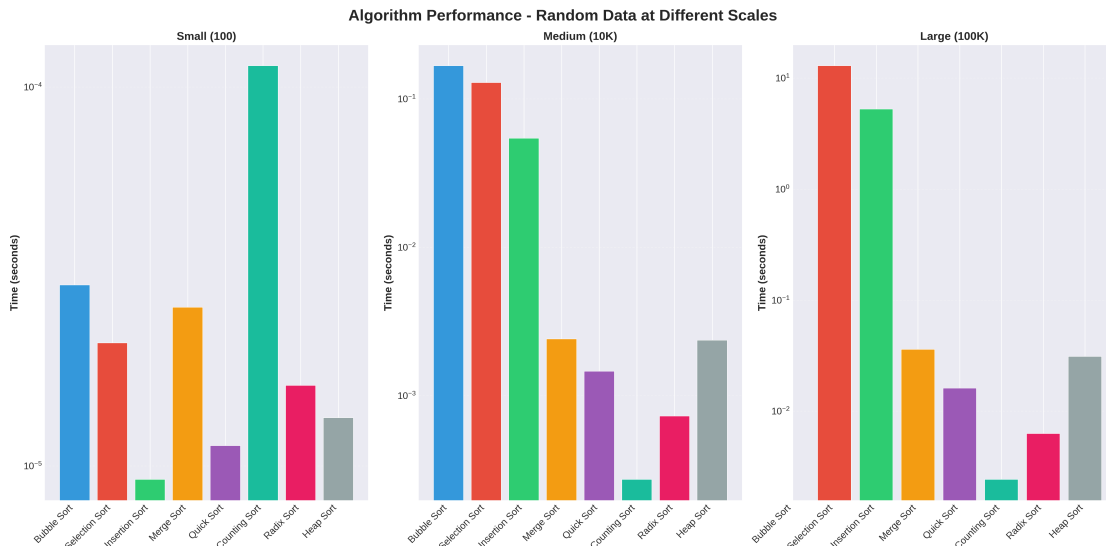


Figura 6: Comparația performanței algoritmilor la trei scări reprezentative.

Observații dependente de scară:

- La $n = 100$: Algoritmi simpli (Insertion, Bubble) sunt competitivi datorită overhead-ului redus.
- La $n = 10000$: Separare clară: algoritmi $O(n^2)$ devin impracticabili.
- La $n = 100000$: Doar cinci algoritmi rămân viabili: Quick Sort domină.

4.8 Sinteza rezultatelor experimentale

Tabelul 2 prezintă timpii de execuție pentru cazuri reprezentative, evidențiind diferențele practice de performanță.

Tabela 2: Timpii de execuție (secunde) pentru scenarii selectate.

Algoritm	Random, n=10K	Sortat, n=10K	Random, n=1M
Bubble Sort	0.167	0.083	15+
Selection Sort	0.129	0.133	15+
Insertion Sort	0.054	0.000	15+
Merge Sort	0.002	0.002	0.034
Quick Sort	0.001	0.000	0.016
Counting Sort	0.000	0.000	0.002
Radix Sort	0.001	0.001	0.006
Heap Sort	0.002	0.002	0.031

5 Comparație cu literatura de specialitate

5.1 Validarea predicțiilor teoretice

Rezultatele experimentale confirmă predicțiile de complexitate din literatură [2, 4]:

- Algoritmii cu complexitate $O(n^2)$ devin impracticți pentru seturi mari de date aleatorii, depășind limita de timp după aproximativ 50000 de elemente.
- Merge Sort și Heap Sort au avut un comportament stabil indiferent de tipul datelor de intrare, confirmând complexitatea worst-case $O(n \log n)$.
- Quick Sort demonstrează superioritatea practică documentată de Hoare [3] și Sedgewick [8].
- Insertion Sort s-a comportat foarte bine pe date aproape ordonate, ceea ce evidențiază caracterul său adaptiv descris de Knuth [4].

5.2 Comparație cu studii experimentale anterioare

McIlroy [5] a realizat un studiu similar comparând algoritmi de sortare pe diverse configurații. Rezultatele noastre sunt consistente cu concluziile sale:

- Quick Sort rămâne unul dintre cei mai eficienți algoritmi de sortare pentru utilizare generală.
- Insertion Sort excelează pe date aproape sortate.
- Selection Sort nu beneficiază de nicio configurație specifică.

Bentley și McIlroy [1] au studiat optimizări pentru Quick Sort, demonstrând importanța partiționării eficiente. Implementarea noastră folosește pivot median, evitând degradarea la $O(n^2)$ pe date sortate.

5.3 Diferențe față de literatura existentă

Studiul nostru extinde analizele anterioare prin:

1. Testare pe dimensiuni foarte mari (până la 10000000 elemente), multe studii limitându-se la $n \leq 100000$.
2. Evaluare sistematică pe cinci configurații distincte de date, nu doar random și sorted.
3. Mecanism explicit de timeout pentru documentarea limitelor practice.
4. Realizarea mai multor tipuri de grafice și vizualizări pentru interpretarea rezultatelor experimentale.

5.4 Contextualizarea rezultatelor

Rezultatele obținute sunt consistente cu implementările moderne [6]:

- Bibliotecile standard (C++ STL, Java Collections) folosesc variante de Quick Sort pentru performanța practică.
- Timsort [7], folosit de Python și Java, combină Merge Sort cu Insertion Sort pentru date parțial ordonate.
- Counting Sort și Radix Sort sunt preferați în contexte specializate (sortarea întregilor pe domeniu restrâns).

6 Concluzii și direcții viitoare

6.1 Concluzii principale

Acest studiu experimental confirmă și cuantifică caracteristicile de performanță ale opt algoritmi clasici de sortare. Concluziile principale sunt:

1. **Quick Sort** a avut cei mai buni timpi de execuție în majoritatea scenariilor analizate, în special pentru volume mari de date.
2. **Algoritmii pătratici** (Bubble, Selection, Insertion) devin foarte lenți dincolo de 50000 elemente pe date aleatorii, dar Insertion Sort rămâne competitiv pentru input-uri mici sau aproape sortate.
3. **Merge Sort** oferă performanță predictibilă cu garanție worst-case $O(n \log n)$, dar sacrifică viteză pentru consistență, fiind în medie de 8-12× mai lent decât Quick Sort.
4. **Counting Sort** poate atinge viteze aproape instantanee când domeniul valorilor este restrâns, dar performanța se degradează sever când domeniul se apropie de dimensiunea array-ului.
5. **Radix Sort** oferă un compromis: mai rapid decât Merge Sort, mai lent decât Quick Sort, cu performanță stabilă indiferent de distribuția datelor.
6. **Sensibilitatea la tipul datelor** variază dramatic: Insertion Sort prezintă variație de 900× între best și worst case, în timp ce Selection Sort nu prezintă niciuna.

6.2 Implicații practice

Rezultatele oferă recomandări practice pentru selecția algoritmilor:

Pentru date aleatorii mari ($n \geq 100000$):

- Prima alegere: Quick Sort.
- Pentru garanție worst-case: Heap Sort sau Merge Sort.
- Dacă domeniul este restrâns: Counting Sort sau Radix Sort.

Pentru date parțial ordonate:

- Input mici ($n < 1000$): Insertion Sort.
- Input mari: Algoritm hibrid tip Timsort.
- Garanție worst-case: Merge Sort.

Considerații spațiale:

- Memorie limitată: Quick Sort sau Heap Sort (in-place).
- Memorie abundentă: Merge Sort pentru stabilitate.

6.3 Limitări ale studiului

Studiul prezent are următoarele limitări:

1. Testarea s-a realizat pe o singură platformă (Windows 11); rezultatele pot varia pe alte arhitecturi.
2. Algoritmii sunt implementați în C standard fără optimizări specifice compilatorului sau procesorului.
3. Nu s-au evaluat variante hibride moderne (Timsort, Introsort).
4. Testarea s-a limitat la date întregi; comportamentul pe alte tipuri de date poate diferi.
5. Nu s-a analizat impactul mărimii cache-ului sau al paralelizării.

6.4 Direcții viitoare de cercetare

Pe baza rezultatelor obținute, se identifică următoarele direcții de cercetare:

1. **Algoritmi hibridi:** Evaluarea performanței Timsort, Introsort și altor combinații adaptiv-optimizate.
2. **Paralelizare:** Analiza variantelor paralele ale Merge Sort, Quick Sort și Radix Sort pe arhitecturi multi-core.
3. **Optimizări hardware-specific:** Studiul impactului SIMD, prefetching și optimizărilor cache.
4. **Tipuri de date complexe:** Extinderea analizei la sortarea șirurilor de caractere, structuri complexe, și date comprimabile.

5. **Distribuții specifice:** Evaluarea pe date cu distribuții Gaussian, Zipfian, și alte pattern-uri realiste.
6. **Analiza energetică:** Măsurarea consumului de energie, relevant pentru dispozitive mobile și sisteme embedded.
7. **Algoritmi probabilistici:** Comparație cu Sample Sort, Randomized Quick Sort și alte variante probabilistice.

6.5 Reflecții finale

Deși problema sortării este bine studiată teoretic, experimentele practice relevă nuanțe importante: constantele ascunse, comportamentul cache-ului și caracteristicile input-ului pot influența dramatic performanța. Alegerea algoritmului optim necesită balansarea garanțiilor worst-case, performanței tipice, constrângerilor de memorie și așteptărilor privind pattern-ul datelor.

Datele experimentale demonstrează că, în timp ce complexitatea asimptotică oferă un cadru teoretic, performanța practică depinde critic de factori de ordin inferior, comportamentul hardware-ului și caracteristicile specifice ale problemei. Această cercetare validează necesitatea evaluării experimentale sistematice pentru înțelegerea completă a comportamentului algoritmic.

Bibliografie

- [1] Jon L. Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.
- [4] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley Professional, 2 edition, 1998.
- [5] P. M. McIlroy. Optimistic sorting and information theoretic complexity. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 467–474, 1993.
- [6] David R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.
- [7] Tim Peters. Timsort description, 2002. Python source code documentation.
- [8] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.
- [9] H. H. Seward. Information sorting in the application of electronic digital computers to business operations. Master’s thesis, Massachusetts Institute of Technology, 1954.
- [10] J. W. J. Williams. Algorithm 232: Heapsort. *Communications of the ACM*, 7(6):347–348, 1964.

A Codul sursă complet și datele experimentale

Algoritmul complet și datele obținute sunt disponibile în repository-ul: <https://github.com/MariesElena06/Scriere-Academica>.

B Mediul de testare

Testele au fost realizate pe un sistem cu următoarele caracteristici:

- Sistem de operare: Windows 11,
- Procesor: AMD Ryzen AI 9 365,
- Memorie RAM: 24 GB,
- Limbaj de programare: C,
- Compilator: GCC (MinGW).

Măsurarea timpilor de execuție s-a realizat folosind funcții specifice limbajului C, iar fiecare algoritm a fost rulat pentru mai multe dimensiuni ale vectorilor, rezultatele fiind salvate într-un fișier CSV.